

ADS 108 – Lista 1- Avaliação de Linguagens de Programação

Exercícios Práticos Comparativos

Legibilidade • Facilidade de Escrita • Confiabilidade

Sobre estes exercícios

Os exercícios a seguir propõem a análise comparativa de trechos de código em diferentes linguagens de programação (Python, Java, C e JavaScript). O objetivo é desenvolver a capacidade crítica de avaliar linguagens segundo três critérios fundamentais:

Legibilidade Facilidade de Escrita Confiabilidade

Exercício 1 Simplicidade e Legibilidade

Critérios avaliados: Legibilidade Facilidade de Escrita

Os códigos abaixo calculam a soma dos elementos de uma lista. Compare as implementações e responda às perguntas.

Python	C
<pre>numeros = [1, 2, 3, 4, 5] soma = sum(numeros) print(f'Soma: {soma}')</pre>	<pre>int numeros[] = {1,2,3,4,5}; int soma = 0, n = 5; for(int i=0; i<n; i++) soma += numeros[i]; printf("Soma: %d\n", soma);</pre>
Java	JavaScript
<pre>int[] numeros = {1,2,3,4,5}; int soma = 0; for (int n : numeros) { soma += n; } System.out.println("Soma: "+soma);</pre>	<pre>const numeros = [1,2,3,4,5]; const soma = numeros.reduce((acc, n) => acc + n, 0); console.log(`Soma: \${soma}`);</pre>

Questões:

1. Qual implementação você considera mais legível para um iniciante? Justifique com base na simplicidade da sintaxe.

2. A versão em Python usa a função `sum()`. Isso facilita ou dificulta a escrita? Como isso se relaciona com o critério de Facilidade de Escrita?
3. Compare o código C com o Python. Quais elementos da sintaxe C aumentam a complexidade visual? Liste ao menos 3 diferenças.
4. O JavaScript usa o método `reduce()`. Isso torna o código mais ou menos legível que o `for-each` do Java? Argumente.

Exercício 2 Verificação de Tipos e Confiabilidade

Critérios avaliados: Confiabilidade Legibilidade

Os trechos abaixo tentam somar um número inteiro com uma string. Observe os resultados e responda.

Python (tipagem dinâmica forte)	Java (tipagem estática forte)
<pre>x = 10 y = "20" resultado = x + y # TypeError em tempo de execução: # unsupported operand type(s)</pre>	<pre>int x = 10; String y = "20"; // ERRO em compilação: // incompatible types: // int + String</pre>
JavaScript (tipagem dinâmica fraca)	C (tipagem estática fraca)
<pre>let x = 10; let y = "20"; let resultado = x + y; // Sem erro! Resultado: // "1020" (concatenação!)</pre>	<pre>int x = 10; char* y = "20"; int r = x + y; // Compila com aviso. // Comportamento indefinido!</pre>

Questões:

5. Por que o Java detecta o erro antes da execução enquanto o Python só detecta durante a execução? Qual característica da linguagem explica isso?
6. O JavaScript não gerou erro, mas produziu um resultado inesperado ('1020'). Por que esse comportamento representa um problema de Confiabilidade?
7. Em qual linguagem o erro seria mais difícil de detectar num sistema grande com milhares de linhas? Justifique.
8. Explique como a verificação de tipos (estática vs. dinâmica) impacta diretamente o critério de Confiabilidade de uma linguagem.
9. Pesquise sobre TypeScript. Como ele resolve o problema observado no JavaScript? Isso melhora a Confiabilidade da linguagem?

Exercício 3 Tratamento de Exceções

Critérios avaliados: Confiabilidade Facilidade de Escrita

Compare como cada linguagem lida com a divisão por zero:

Python	Java
<pre>try:</pre>	<pre>try {</pre>

<pre> resultado = 10 / 0 except ZeroDivisionError as e: print(f'Erro: {e}') finally: print('Finalizado') </pre>	<pre> int r = 10 / 0; } catch (ArithmeticException e) { System.out.println("Erro: "+e); } finally { System.out.println("Fim"); } </pre>
C (sem exceções nativas)	JavaScript
<pre> int dividir(int a, int b) { if (b == 0) { printf("Erro!\n"); return -1; } return a / b; } </pre>	<pre> try { let r = 10 / 0; // Atenção: retorna Infinity! if (!isFinite(r)) throw new Error('Div 0'); } catch (e) { console.log(e); } </pre>

Questões:

10. A linguagem C não possui mecanismo nativo de exceções. Como isso impacta a Confiabilidade de programas escritos em C?
11. Compare a sintaxe do try-catch em Python, Java e JavaScript. Qual você considera mais fácil de escrever (Facilidade de Escrita)? Qual é mais clara (Legibilidade)?
12. No JavaScript, a divisão por zero não lança exceção — retorna Infinity. Isso é um problema de Confiabilidade? Explique o risco em um sistema financeiro.
13. Em qual linguagem você preferiria desenvolver um sistema bancário crítico? Baseie sua resposta exclusivamente nos critérios de Confiabilidade vistos neste exercício.

Exercício 4 Suporte à Abstração e Expressividade

Critérios avaliados: Facilidade de Escrita Confiabilidade

Os códigos abaixo filtram números pares de uma lista e retornam seus quadrados:

Python — list comprehension	Java — Streams API
<pre> nums = [1,2,3,4,5,6,7,8] resultado = [x**2 for x in nums if x % 2 == 0] print(resultado) </pre>	<pre> List<Integer> nums = Arrays.asList(1,2,3,4,5,6,7,8); List<Integer> resultado = nums.stream() .filter(x -> x%2==0) .map(x -> x*x) .collect(toList()); </pre>
C — sem abstração de alto nível	JavaScript — Array methods
<pre> int nums[]={1,2,3,4,5,6,7,8}; int res[8]; int j=0; for(int i=0;i<8;i++) { if (nums[i]%2==0) res[j++]=nums[i]*nums[i]; } </pre>	<pre> const nums=[1,2,3,4,5,6,7,8]; const resultado = nums .filter(x => x%2===0) .map(x => x**2); console.log(resultado); </pre>

Questões:

14. A versão em C requer controle manual do índice j e do tamanho do array. Como isso afeta a Confiabilidade em comparação com Python e JavaScript?

15. A sintaxe de list comprehension do Python é considerada expressiva. Explique o que 'expressividade' significa neste contexto e por que ela impacta a Facilidade de Escrita.
16. Compare o Java com o JavaScript na solução deste problema. Qual requer mais código? Isso é um indicador de qual critério de avaliação?
17. Abstração significa esconder detalhes de implementação. Identifique em quais linguagens o programador precisa 'pensar em baixo nível' para resolver este problema.
18. Escreva você mesmo a solução para calcular a soma dos cubos dos números ímpares entre 1 e 20 nas 4 linguagens. Compare o esforço de escrita.

Exercício 5 Apelidos Restritos (Aliasing) e Projeto de Sintaxe

Critérios avaliados: Confiabilidade Legibilidade

Apelidos (aliasing) ocorrem quando dois identificadores distintos referenciam a mesma posição de memória. Compare os comportamentos:

Python — referência em listas	Java — referência em objetos
<pre>lista_a = [1, 2, 3] lista_b = lista_a # alias! lista_b.append(4) print(lista_a) # [1,2,3,4] # lista_a foi modificada # sem intenção explícita!</pre>	<pre>int[] a = {1, 2, 3}; int[] b = a; // alias! b[0] = 99; System.out.println(a[0]); // 99 // a[0] foi alterado! // sem aviso do compilador</pre>
C — ponteiros (aliasing explícito)	Python — tipos imutáveis (sem alias)
<pre>int x = 10; int *p1 = &x; int *p2 = &x; // alias! *p1 = 20; printf("%d", *p2); // 20 // Risco de erro sutil em C</pre>	<pre># Inteiros são imutáveis em Python a = 10 b = a # copia valor b = 20 # cria novo objeto print(a) # ainda 10 # Sem problema de aliasing!</pre>

Questões:

19. Explique com suas palavras o que é aliasing e por que ele pode ser considerado um problema de Confiabilidade.
20. Por que tipos imutáveis em Python (int, str, tuple) não sofrem com o problema de aliasing, enquanto listas sofrem?
21. Em C, o programador usa ponteiros explicitamente, o que torna o aliasing visível no código. Isso é vantagem ou desvantagem em termos de Legibilidade?
22. Linguagens funcionais como Haskell proíbem mutação de dados (imutabilidade total). Como isso eliminaria o problema de aliasing? Há desvantagens?
23. Você está depurando um sistema em Java e encontra um bug onde um objeto é modificado inesperadamente. O aliasing poderia ser a causa? Descreva como você investigaria.